What is Amazon S3?

What is Amazon S3?

Amazon Simple Storage Service is storage for the Internet. It is designed to make web-scale computing easier for developers.

Amazon S3 has a simple web services interface that you can use to store and retrieve any amount of data, at any time, from anywhere on the web. It gives any developer access to the same highly scalable, reliable, fast, inexpensive data storage infrastructure that Amazon uses to run its own global network of web sites. The service aims to maximize benefits of scale and to pass those benefits on to developers.

This guide explains the core concepts of Amazon S3, such as buckets, access points, and objects, and how to work with these resources using the Amazon S3 application programming interface (API).

Advantages of using Amazon S3

Amazon S3 is intentionally built with a minimal feature set that focuses on simplicity and robustness. Following are some of the advantages of using Amazon S3:

- **Creating buckets** – Create and name a bucket that stores data. Buckets are the fundamental containers in Amazon S3 for data storage.
- **Storing data** – Store an infinite amount of data in a bucket. Upload as many objects as you like into an Amazon S3 bucket. Each object can contain up to 5 TB of data. Each object is stored and retrieved using a unique developer-assigned key.
- **Downloading data** – Download your data or enable others to do so. Download your data anytime you like, or allow others to do the same.
- **Permissions** – Grant or deny access to others who want to upload or download data into your Amazon S3 bucket. Grant upload and download permissions to three types of users. Authentication mechanisms can help keep data secure from unauthorized access.
- **Standard interfaces** – Use standards-based REST and SOAP interfaces designed to work with any internet-development toolkit.

Amazon S3 concepts

Buckets

A bucket is a container for objects stored in Amazon S3. Every object is contained in a bucket. For example, if the object named *photos/puppy.jpg* is stored in the awsexamplebucket1 bucket in the US West (Oregon) Region, then it is addressable using the URL *https://awsexamplebucket1.s3.us-west-2.amazonaws.com/photos/puppy.jpg.*

Buckets serve several purposes:

- They organize the Amazon S3 namespace at the highest level.
- They identify the account responsible for storage and data transfer charges.
- They play a role in access control.
- They serve as the unit of aggregation for usage reporting.

You can configure buckets so that they are created in a specific AWS Region.

Objects

Objects are the fundamental entities stored in Amazon S3. Objects consist of object data and metadata. The data portion is opaque to Amazon S3. The metadata is a set of name-value pairs that describe the object. These include some default metadata, such as the date last modified, and standard HTTP metadata, such as `Content-Type`. You can also specify custom metadata at the time the object is stored.

An object is uniquely identified within a bucket by a key (name) and a version ID.

Keys

A key is the unique identifier for an object within a bucket. Every object in a bucket has exactly one key. The combination of a bucket, key, and version ID uniquely identify each object. So you can think of Amazon S3 as a basic data map between "bucket + key + version" and the object itself. Every object in Amazon S3 can be uniquely addressed through the combination of the web service endpoint, bucket name, key, and optionally, a version. For example, in the URL *https://doc.s3.amazonaws.com/2006-03-01/AmazonS3.wsdl*, *"doc"* is the name of the bucket and *"2006-03-01/AmazonS3.wsdl"* is the key.

Regions

You can choose the geographical AWS Region where Amazon S3 will store the buckets that you create. You might choose a Region to optimize latency, minimize costs, or address regulatory requirements. Objects stored in a Region never leave the Region unless you explicitly transfer them to another Region. For example, objects stored in the Europe (Ireland) Region never leave it.

Amazon S3 data consistency model

Amazon S3 provides strong read-after-write consistency for PUTs and DELETEs of objects in your Amazon S3 bucket in all AWS Regions. This applies to both writes to new objects as well as PUTs that overwrite existing objects and DELETEs. In addition, read operations on Amazon S3 Select, Amazon S3 Access Control Lists, Amazon S3 Object Tags, and object metadata (e.g. HEAD object) are strongly consistent.

Updates to a single key are atomic. For example, if you PUT to an existing key from one thread and perform a GET on the same key from a second thread concurrently, you will get either the old data or the new data, but never partial or corrupt data.

Amazon S3 data consistency model

Amazon S3 achieves high availability by replicating data across multiple servers within AWS data centers. If a PUT request is successful, your data is safely stored. Any read (GET or LIST) that is initiated following the receipt of a successful PUT response will return the data written by the PUT. Here are examples of this behavior:

- A process writes a new object to Amazon S3 and immediately lists keys within its bucket. The new object will appear in the list.
- A process replaces an existing object and immediately tries to read it. Amazon S3 will return the new data.
- A process deletes an existing object and immediately tries to read it. Amazon S3 will not return any data as the object has been deleted.
- A process deletes an existing object and immediately lists keys within its bucket. The object will not appear in the listing.

Amazon S3 data consistency model

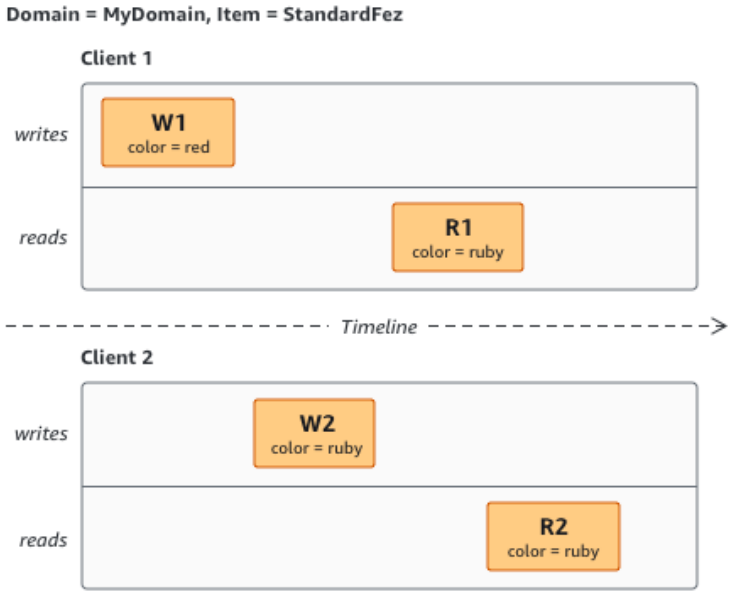
Bucket configurations have an eventual consistency model. Specifically:

- If you delete a bucket and immediately list all buckets, the deleted bucket might still appear in the list.
- If you enable versioning on a bucket for the first time, it might take a short amount of time for the change to be fully propagated. We recommend that you wait for 15 minutes after enabling versioning before issuing write operations (PUT or DELETE) on objects in the bucket.

Concurrent applications

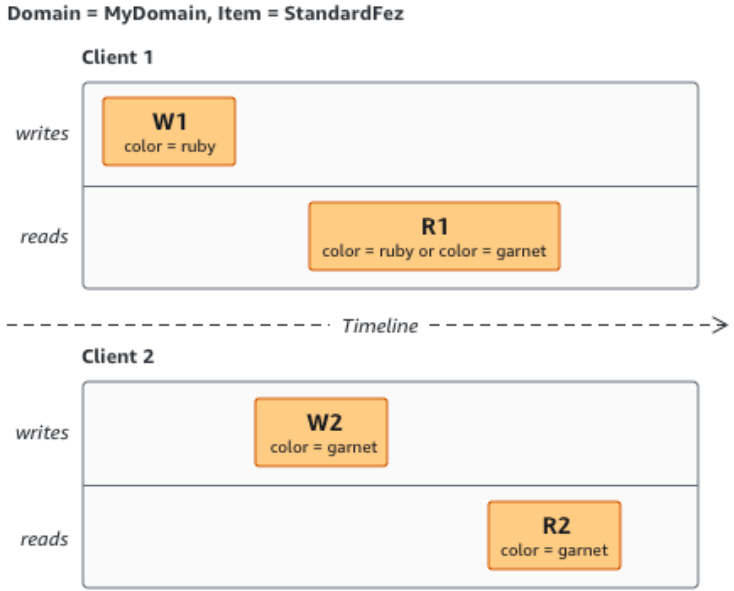
This section provides examples of behavior to be expected from Amazon S3 when multiple clients are writing to the same items.

In this example, both W1 (write 1) and W2 (write 2) complete before the start of R1 (read 1) and R2 (read 2). Because S3 is strongly consistent, R1 and R2 both return `color = ruby`.



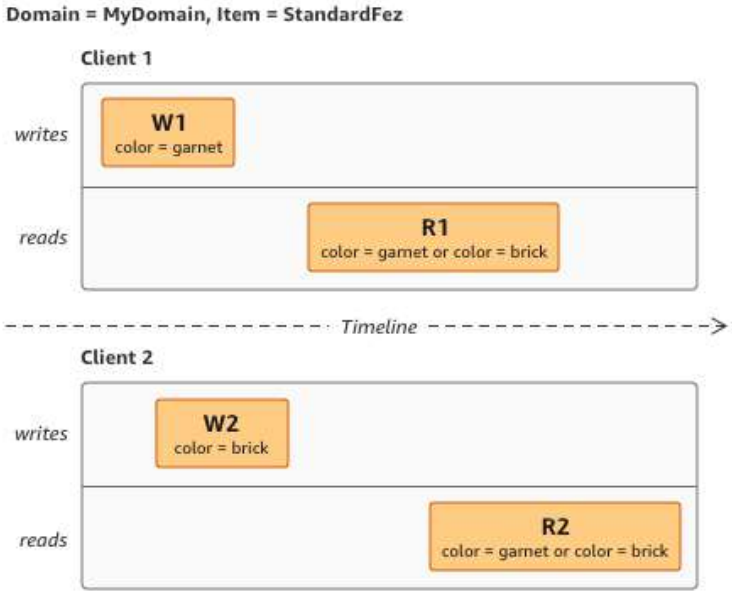
Concurrent applications

In the next example, W2 does not complete before the start of R1. Therefore, R1 might return `color = ruby` or `color = garnet`. However, since W1 and W2 finish before the start of R2, R2 returns `color = garnet`.



Concurrent applications

In the last example, W2 begins before W1 has received an acknowledgement. Therefore, these writes are considered concurrent. Amazon S3 internally uses last-writer-wins semantics to determine which write takes precedence. However, the order in which Amazon S3 receives the requests and the order in which applications receive acknowledgements cannot be predicted due to factors such as network latency. For example, W2 might be initiated by an Amazon EC2 instance in the same region while W1 might be initiated by a host that is further away. The best way to determine the final value is to perform a read after both writes have been acknowledged.



Amazon S3 features

Storage classes

Amazon S3 offers a range of storage classes designed for different use cases. These include Amazon S3 STANDARD for general-purpose storage of frequently accessed data, Amazon S3 STANDARD_IA for long-lived, but less frequently accessed data, and S3 Glacier for long-term archive.

Bucket policies

Bucket policies provide centralized access control to buckets and objects based on a variety of conditions, including Amazon S3 operations, requesters, resources, and aspects of the request (for example, IP address). The policies are expressed in the *access policy language* and enable centralized management of permissions. The permissions attached to a bucket apply to all of the bucket's objects that are owned by the bucket owner account.

Bucket policies

Both individuals and companies can use bucket policies. When companies register with Amazon S3, they create an account. Thereafter, the company becomes synonymous with the account. Accounts are financially responsible for the AWS resources that they (and their employees) create. Accounts have the power to grant bucket policy permissions and assign employees permissions based on a variety of conditions. For example, an account could create a policy that gives a user write access:

- To a particular S3 bucket
- From an account's corporate network
- During business hours

An account can grant one user limited read and write access, but allow another to create and delete buckets also. An account could allow several field offices to store their daily reports in a single bucket. It could allow each office to write only to a certain set of names (for example, "Nevada/*" or "Utah/*") and only from the office's IP address range.

Bucket policies

Unlike access control lists (described later), which can add (grant) permissions only on individual objects, policies can either add or deny permissions across all (or a subset) of objects within a bucket. With one request, an account can set the permissions of any number of objects in a bucket. An account can use wildcards (similar to regular expression operators) on Amazon Resource Names (ARNs) and other values. The account could then control access to groups of objects that begin with a common prefix or end with a given extension, such as *.html*.

Bucket policies

Only the bucket owner is allowed to associate a policy with a bucket. Policies (written in the access policy language) allow or deny requests based on the following:

- Amazon S3 bucket operations, and object operations (such as `PUT Object`, or `GET Object`)
- Requester
- Conditions specified in the policy

An account can control access based on specific Amazon S3 operations, such as `GetObject`, `GetObjectVersion`, `DeleteObject`, or `DeleteBucket`.

The conditions can be such things as IP addresses, IP address ranges in CIDR notation, dates, user agents, HTTP referrer, and transports (HTTP and HTTPS).

AWS identity and access management

You can use AWS Identity and Access Management (IAM) to manage access to your Amazon S3 resources.

For example, you can use IAM with Amazon S3 to control the type of access a user or group of users has to specific parts of an Amazon S3 bucket your AWS account owns.

Access control lists

You can control access to each of your buckets and objects using an access control list (ACL).

Versioning

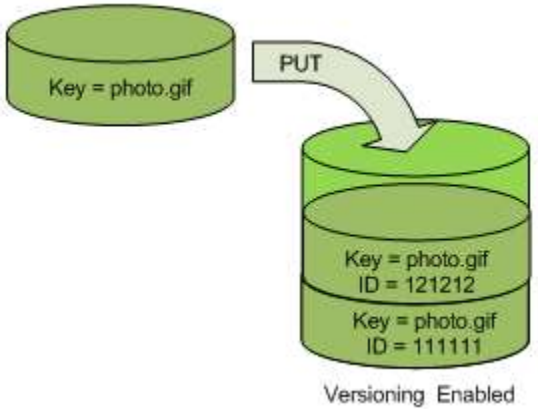
Use Amazon S3 Versioning to keep multiple versions of an object in one bucket. For example, you could store `my-image.jpg` (version 111111) and `my-image.jpg` (version 222222) in a single bucket. S3 Versioning protects you from the consequences of unintended overwrites and deletions. You can also use it to archive objects so that you have access to previous versions.

To customize your data retention approach and control storage costs, use object versioning with Object lifecycle management.

You must explicitly enable S3 Versioning on your bucket. By default, S3 Versioning is disabled. Regardless of whether you have enabled Versioning, each object in your bucket has a version ID. If you have not enabled Versioning, Amazon S3 sets the value of the version ID to null. If S3 Versioning is enabled, Amazon S3 assigns a version ID value for the object. This value distinguishes it from other versions of the same key.

Versioning

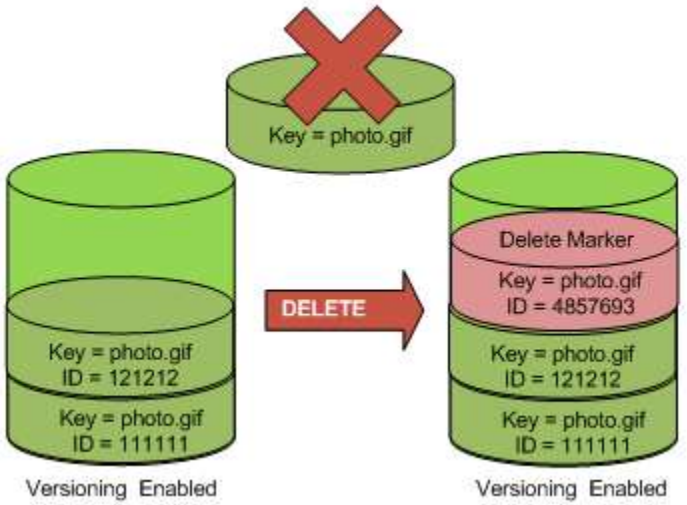
When you PUT an object in a versioning-enabled bucket, the noncurrent version is not overwritten. The following figure shows that when a new version of photo.gif is PUT into a bucket that already contains an object with the same name, the original object (ID = 111111) remains in the bucket, Amazon S3 generates a new version ID (121212), and adds the newer version to the bucket.



Versioning

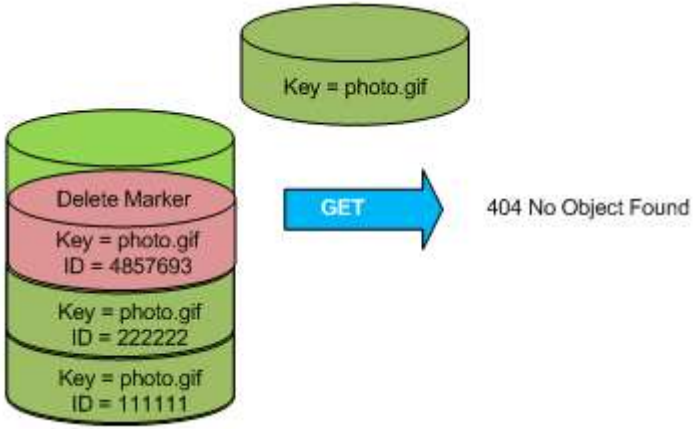
This functionality prevents you from accidentally overwriting or deleting objects and affords you the opportunity to retrieve a previous version of an object.

When you DELETE an object, all versions remain in the bucket and Amazon S3 inserts a delete marker, as shown in the following figure.



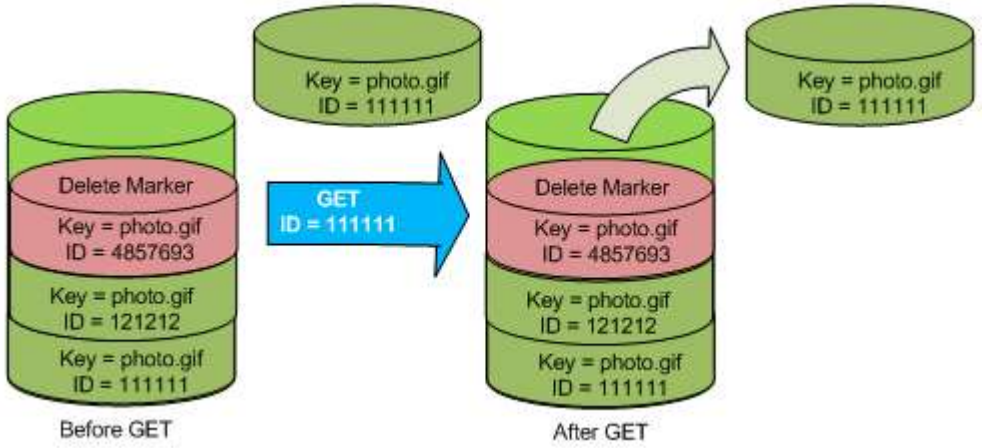
Versioning

The delete marker becomes the current version of the object. By default, GET requests retrieve the most recently stored version. Performing a simple GET Object request when the current version is a delete marker returns a 404 Not Found error, as shown in the following figure.



Versioning

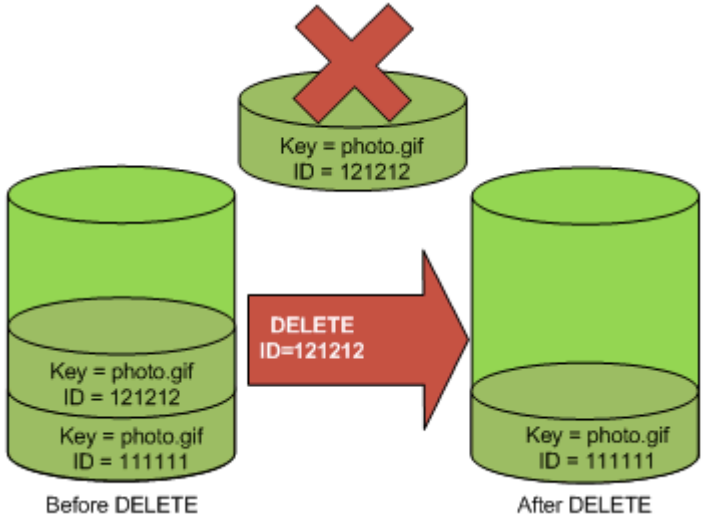
However, you can GET a noncurrent version of an object by specifying its version ID. In the following figure, you GET a specific object version, 111111. Amazon S3 returns that object version even though it's not the current version.



Versioning

You can permanently delete an object by specifying the version you want to delete. Only the owner of an Amazon S3 bucket can permanently delete a version. The following figure shows how DELETE versionId permanently deletes an object from a bucket and that Amazon S3 doesn't insert a delete marker.

You can add additional security by configuring a bucket to enable MFA (multi-factor authentication) Delete. When you do, the bucket owner must include two forms of authentication in any request to delete a version or change the versioning state of the bucket.



Operations

Following are the most common operations that you'll run through the API.

Common operations

- **Create a bucket** – Create and name your own bucket in which to store your objects.
- **Write an object** – Store data by creating or overwriting an object. When you write an object, you specify a unique key in the namespace of your bucket. This is also a good time to specify any access control you want on the object.
- **Read an object** – Read data back. You can download the data via HTTP or BitTorrent.
- **Delete an object** – Delete some of your data.
- **List keys** – List the keys contained in one of your buckets. You can filter the key list based on a prefix.

Amazon S3 application programming interfaces (API)

The Amazon S3 architecture is designed to be programming language-neutral, using AWS supported interfaces to store and retrieve objects.
Amazon S3 provides a REST and a SOAP interface. They are similar, but there are some differences. For example, in the REST interface, metadata is returned in HTTP headers. Because we only support HTTP requests of up to 4 KB (not including the body), the amount of metadata you can supply is restricted.

The REST interface

The REST API is an HTTP interface to Amazon S3. Using REST, you use standard HTTP requests to create, fetch, and delete buckets and objects.
You can use any toolkit that supports HTTP to use the REST API. You can even use a browser to fetch objects, as long as they are anonymously readable.
The REST API uses the standard HTTP headers and status codes, so that standard browsers and toolkits work as expected. In some areas, we have added functionality to HTTP (for example, we added headers to support access control). In these cases, we have done our best to add the new functionality in a way that matched the style of standard HTTP usage.

The SOAP interface

The SOAP API provides a SOAP 1.1 interface using document literal encoding. The most common way to use SOAP is to download the WSDL, use a SOAP toolkit such as Apache Axis or Microsoft .NET to create bindings, and then write code that uses the bindings to call Amazon S3.

Paying for Amazon S3

Pricing for Amazon S3 is designed so that you don't have to plan for the storage requirements of your application. Most storage providers force you to purchase a predetermined amount of storage and network transfer capacity: If you exceed that capacity, your service is shut off or you are charged high overage fees. If you do not exceed that capacity, you pay as though you used it all.

Amazon S3 charges you only for what you actually use, with no hidden fees and no overage charges. This gives developers a variable-cost service that can grow with their business while enjoying the cost advantages of the AWS infrastructure.

Before storing anything in Amazon S3, you must register with the service and provide a payment method that is charged at the end of each month. There are no setup fees to begin using the service. At the end of the month, your payment method is automatically charged for that month's usage.

Q&A